
Started on Saturday, 28 March 2026, 12:05 PM

State Finished

Completed on Saturday, 28 March 2026, 1:15 PM

Time taken 1 hour 9 mins

Grade 10.00 out of 10.00 (100%)

Question 1 | Correct Mark 1.00 out of 1.00

Consider the below words as key words and check the given input is key word or not.

keywords: {break, case, continue, default, defer, else, for, func, goto, if, map, range, return, struct, type, var}

Input format:

Take string as an input from stdin.

Output format:

Print the word is key word or not.

Example Input:

break

Output:

break is a keyword

Example Input:

IF

Output:

IF is not a keyword

For example:

Input	Result
break	break is a keyword
IF	IF is not a keyword

Answer: (penalty regime: 0 %)

```
1 keywords = ["break", "case", "continue", "default", "defer", "else", "for", "func", "goto", "if", "map", "range", "return", "st
2 word = input()
3 if word in keywords:
4     print(f"{word} is a keyword")
5 else:
6     print(f"{word} is not a keyword")
```

	Input	Expected	Got	
✓	break	break is a keyword	break is a keyword	✓
✓	IF	IF is not a keyword	IF is not a keyword	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 2 | Correct Mark 1.00 out of 1.00

A pangram is a sentence where every letter of the English alphabet appears at least once.

Given a string sentence containing only lowercase English letters, return true if sentence is a pangram, or false otherwise.

Example 1:

Input:

thequickbrownfoxjumpsoverthelazydog

Output:

true

Explanation: sentence contains at least one of every letter of the English alphabet.

Example 2:

Input:

arvijayakumar

Output: false

Constraints:

1 <= sentence.length <= 1000

sentence consists of lowercase English letters.

For example:

Test	Result
print(checkPangram('thequickbrownfoxjumpsoverthelazydog'))	true
print(checkPangram('arvijayakumar'))	false

Answer: (penalty regime: 0 %)

[Reset answer](#)

```
1 def checkPangram(s):
2     alpha="abcdefghijklmnopqrstuvwxyz"
3     for c in alpha:
4         if c not in s:
5             return "false"
6     return "true"
7
```

	Test	Expected	Got	
✓	print(checkPangram('thequickbrownfoxjumpsoverthelazydog'))	true	true	✓
✓	print(checkPangram('arvijayakumar'))	false	false	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.



Question 3 | Correct Mark 1.00 out of 1.00

Given a string *S* which is of the format *USERNAME@DOMAIN.EXTENSION*, the program must print the *EXTENSION*, *DOMAIN*, *USERNAME* in the reverse order.

Input Format:

The first line contains *S*.

Output Format:

The first line contains *EXTENSION*.

The second line contains *DOMAIN*.

The third line contains *USERNAME*.

Boundary Condition:

1 <= Length of *S* <= 100

Example Input/Output 1:

Input:

abcd@gmail.com

Output:

com

gmail

abcd

For example:

Input	Result
arvijayakumar@rajalakshmi.edu.in	edu.in rajalakshmi arvijayakumar

Answer: (penalty regime: 0 %)

```
1 email=input()
2 part=email.split('@')
3 username=part[0]
4 domain=part[1]
5 domains=domain.split('.',1)
6 dom=domains[0]
7 extension=domains[1]
8 print(f"{extension}\n{dom}\n{username}")
```

	Input	Expected	Got	
✓	abcd@gmail.com	com gmail abcd	com gmail abcd	✓
✓	arvijayakumar@rajalakshmi.edu.in	edu.in rajalakshmi arvijayakumar	edu.in rajalakshmi arvijayakumar	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.



Question 4 | Correct Mark 1.00 out of 1.00

Find if a String2 is substring of String1. If it is, return the index of the first occurrence. else return -1.

Sample Input 1

this test123string

123

Sample Output 1

8

Answer: (penalty regime: 0 %)

```
1 a=input()
2 b=input()
3 index=a.find(b)
4 print(index)
```

	Input	Expected	Got	
✓	this test123string 123	8	8	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 5 | Correct Mark 1.00 out of 1.00

The program must accept **N** series of keystrokes as string values as the input. The character ^ represents undo action to clear the last entered keystroke. The program must print the string typed after applying the undo operations as the output. If there are no characters in the string then print **-1** as the output.

Boundary Condition(s):

1 <= N <= 100

1 <= Length of each string <= 100

Input Format:

The first line contains the integer N.

The next N lines contain a string on each line.

Output Format:

The first N lines contain the string after applying the undo operations.

Example Input/Output 1:

Input:

```
3
Hey ^ goooo^^glee^
lucke^y ^charr^ms
ora^^nge^^^^
```

Output:

```
Hey google
luckycharms
-1
```

Answer: (penalty regime: 0 %)

```
1 s=int(input())
2 for _ in range(s) :
3     s=input().strip()
4     stack=[]
5     for char in s :
6         if char=='^' :
7             if stack :
8                 stack.pop()
9             else:
10                stack.append(char)
11 if stack :
12     result=''.join(stack)
13     print(result)
14 else:
```

```
15 | .....  
    | print("-1")
```

	Input	Expected	Got	
✓	3 Hey ^ goooo^glee^ lucke^y ^charr^ms ora^^nge^^^^	Hey google luckycharms -1	Hey google luckycharms -1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 6 | Correct Mark 1.00 out of 1.00

Write a Python program to get one string and reverses a string. The input string is given as an array of characters `char[]` .

You may assume all the characters consist of printable ascii characters.

Example 1:

Input :

hello

Output :

olleh

Example 2:

Input :

Hannah

Output :

hannaH

Answer: (penalty regime: 0 %)

```
1 a= input()
2 print(a[::-1])
```

	Input	Expected	Got	
✓	hello	olleh	olleh	✓
✓	Hannah	hannaH	hannaH	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 7 | Correct Mark 1.00 out of 1.00

Given a string *s* containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

An input string is valid if:

Open brackets must be closed by the same type of brackets.

Open brackets must be closed in the correct order.

Constraints:

$1 \leq s.length \leq 10^4$

s consists of parentheses only '()[]{}'.

For example:

Test	Result
<code>print(ValidParenthesis("()"))</code>	true
<code>print(ValidParenthesis("()[]{}"))</code>	true
<code>print(ValidParenthesis("[)")</code>	false

Answer: (penalty regime: 0 %)

Reset answer

```

1 def ValidParenthesis(s):
2     while '()' in s or '{}' in s or '[]' in s:
3         s=s.replace('()', '').replace('{}', '').replace('[]', '')
4     if len(s)==0:
5         return "true"
6     else:
7         return "false"

```

	Test	Expected	Got	
✓	<code>print(ValidParenthesis("()"))</code>	true	true	✓
✓	<code>print(ValidParenthesis("()[]{}"))</code>	true	true	✓
✓	<code>print(ValidParenthesis("[)")</code>	false	false	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 8 | Correct Mark 1.00 out of 1.00

Given a **non-empty** string `s` and an abbreviation `abbr`, return whether the string matches with the given abbreviation.

A string such as "word" contains only the following valid abbreviations:

```
["word", "1ord", "w1rd", "wo1d", "wor1", "2rd", "w2d", "wo2", "1o1d", "1or1", "w1r1", "1o2", "2r1", "3d", "w3", "4"]
```

Notice that only the above abbreviations are valid abbreviations of the string "word". Any other string is not a valid abbreviation of "word".

Note:

Assume `s` contains only lowercase letters and `abbr` contains only lowercase letters and digits.

Example 1:**Input**

```
internationalization
```

```
i12iz4n
```

Output

```
true
```

Explanation

Given `s` = "internationalization", `abbr` = "i12iz4n":

Return true.

Example 2:**Input**

```
apple
```

```
a2e
```

Output

```
false
```

Explanation

Given **s** = "apple", **abbr** = "a2e":

Return false.

Answer: (penalty regime: 0 %)

```

1 s=input()
2 abbr=(input())
3 i=j=0
4 while i<len(s) and j<len(abbr):
5     if abbr[j].isdigit():
6         num=0
7         while j<len(abbr) and abbr[j].isdigit():
8             num=num*10+int(abbr[j])
9             j+=1
10        i+=num
11    else:
12        if s[i]!=abbr[j]:
13            print("false")
14            exit()
15        i+=1
16        j+=1
17 if i==len(s) and j==len(abbr):
18     print("true")
19 else:
20     print("false")

```

	Input	Expected	Got	
✓	internationalization i12iz4n	true	true	✓
✓	apple a2e	false	false	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 9 | Correct Mark 1.00 out of 1.00

Given a string, determine if it is a palindrome, considering only alphanumeric characters and ignoring cases.

Note: For the purpose of this problem, we define empty string as valid palindrome.

Example 1:

Input:

A man, a plan, a canal: Panama

Output:

1

Example 2:

Input:

race a car

Output:

0

Constraints:

- `s` consists only of printable ASCII characters.

Answer: (penalty regime: 0 %)

```
1 s=input()
2 t=''.join(c.lower() for c in s if c.isalnum())
3 print(int(t==t[::-1]))
```

	Input	Expected	Got	
✓	A man, a plan, a canal: Panama	1	1	✓
✓	race a car	0	0	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 10 | Correct Mark 1.00 out of 1.00

Assume that the given string has enough memory.

Don't use any extra space(IN-PLACE)

Sample Input 1

a2b4c6

Sample Output 1

aabbbbcccccc

Answer: (penalty regime: 0 %)

```

1 s=input()
2 i=0
3 result=""
4 while i<len(s) :
5     char=s[i]
6     i+=1
7     num=0
8     while i<len(s) and s[i].isdigit() :
9         num=num*10+int(s[i])
10        i+=1
11        result+=char*num
12 print(result)

```

	Input	Expected	Got	
✓	a2b4c6	aabbbbcccccc	aabbbbcccccc	✓
✓	a12b3d4	aaaaaaaaaabbddddd	aaaaaaaaaabbddddd	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.